

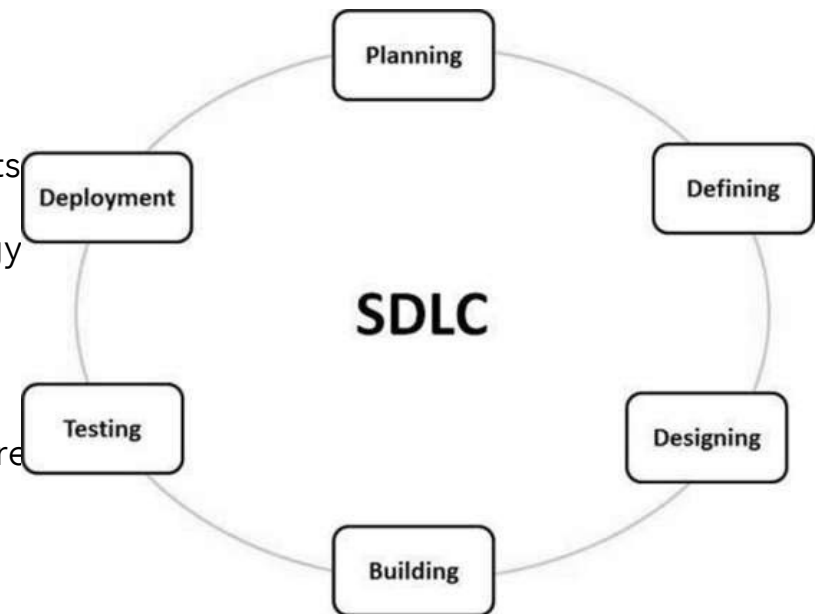


SDLC

SDLC

- The software development life cycle (SDLC) is a framework defining tasks performed at each step in the software development process. SDLC is a structure followed by a development team within the software organization. It consists of a detailed plan describing how to develop, maintain and replace specific software. The life cycle defines a methodology for improving the quality of software and the overall development process.

The software development life cycle is also known as the software development process.



Importance of SDLC



- The SDLC is NOT Optional
- “The earlier the better, but not too late and never ignore.” Without threat modeling, your security is a gamble—and in today’s business environment, you’re sure to lose.

Why MS SDLC?



- Very well documented
- Widely known
- Source:
<https://www.microsoft.com/en-us/securityengineering/sdl/>



Microsoft



Legendary Bill Gates E-Mail



-----Original Message-----

From: Bill Gates

Sent: Tuesday, January 15, 2002 5:22 PM

To: Microsoft and Subsidiaries: All FTE

Subject: Trustworthy computing

Every few years I have sent out a memo talking about the highest priority for Microsoft. Two years ago, it was the kickoff of our .NET strategy. Before that, it was several memos about the importance of the Internet to our future and the ways we could make the Internet truly useful for people. Over the last year it has become clear that ensuring .NET is a platform for Trustworthy Computing is more important than any other part of our work. If we don't do this, people simply won't be willing -- or able -- to take advantage of all the other great work we do. Trustworthy Computing is the highest priority for all the work we are doing. We must lead the industry to a whole new level of Trustworthiness in computing....

...

→ <https://www.itprotoday.com/strategy/complete-text-bill-gates-trustworthy-computing-memo>

→ Read it 5'

<https://www.itprotoday.com/strategy/complete-text-bill-gates-trustworthy-computing-memo>

Mandatory: Which apps are required to follow SDL?



- **Any release** commonly used or deployed within an enterprise, business, or organization
- **Any release** that regularly stores, processes, or communicates PII (as defined in Microsoft Privacy Guidelines for Developing Software Products and Services) or other sensitive customer information
- **Any release** that regularly touches or listens on the Internet or other networks
- **Any release** that accepts and/or processes data from an unauthenticated source
- **Any functionality** that parses any file type that is not protected, (i.e. not limited to system administrators)
- **Any release** that contains ActiveX and/or COM controls
- **All** Microsoft, MSN and Live.com online services that are used by external customers and hosted in the MSN environment

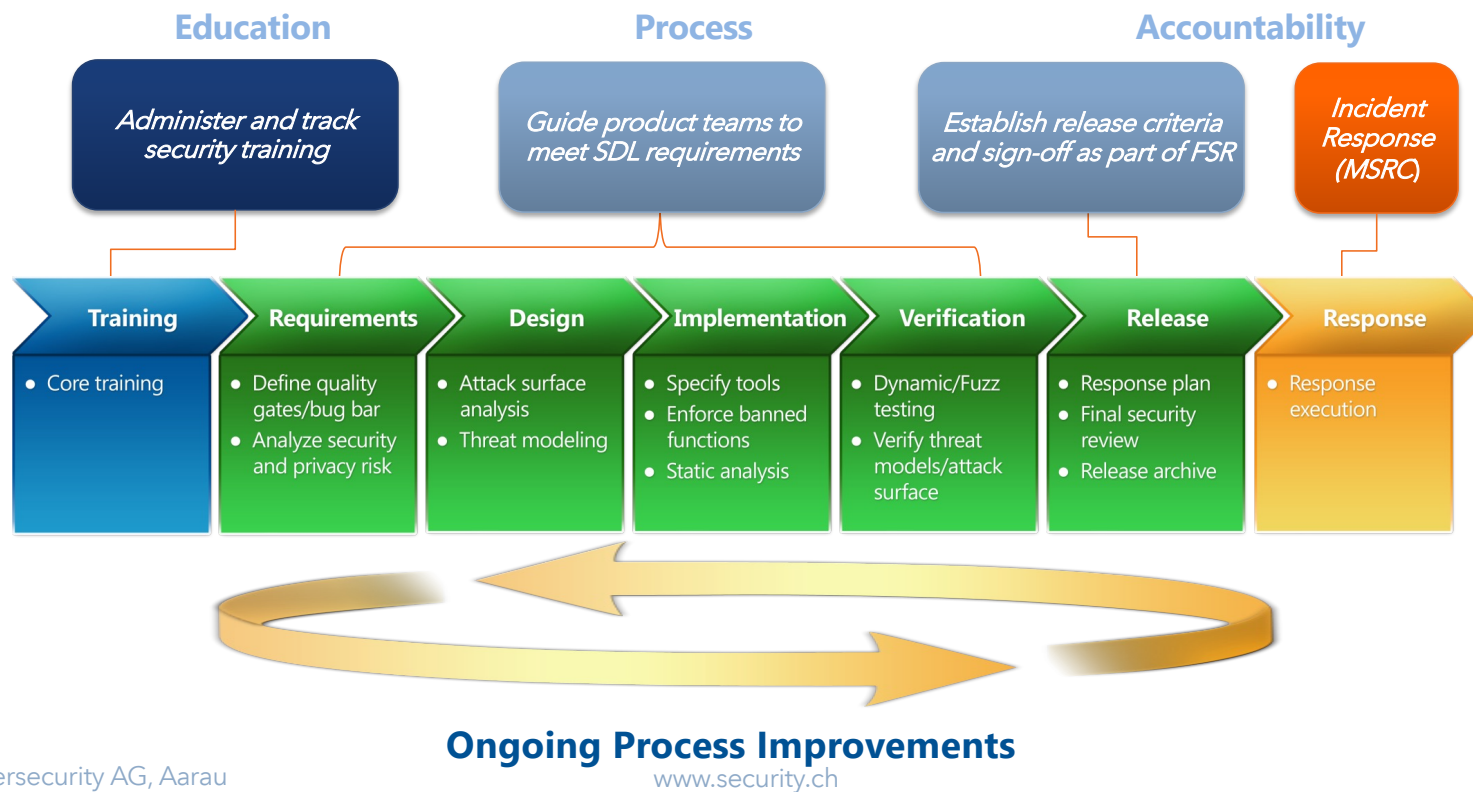
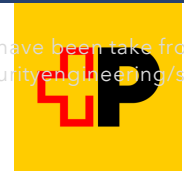


What is Microsoft doing about the threat?

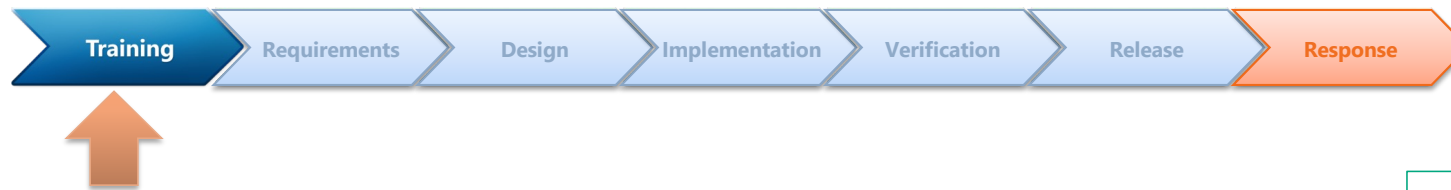
<https://www.microsoft.com/en-us/securityengineering/sdl/threatmodeling>

Working to protect our users...

The following slides concerning MSSDLC have been taken from:
<https://www.microsoft.com/en-us/securityengineering/sdl/>



Pre-SDL Requirements: Security Training

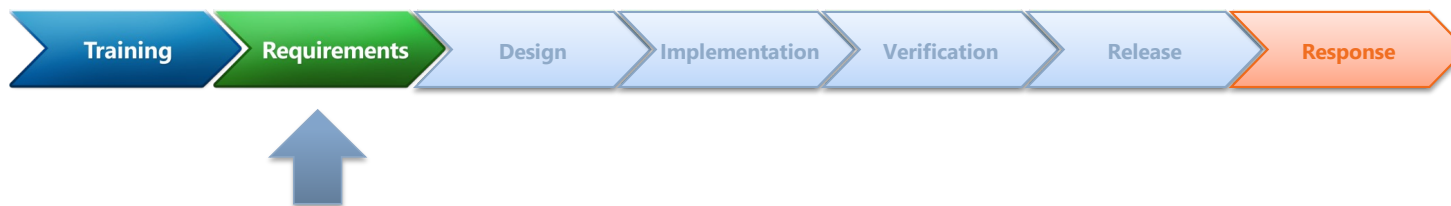


Assess organizational knowledge on security and privacy – establish training program as necessary

- Establish training criteria
 - Content covering secure design, development, test and privacy
- Establish minimum training frequency
 - Employees must attend n classes per year
- Establish minimum acceptable group training thresholds
 - Organizational training targets (e.g. 80% of all technical personnel trained prior to product RTM)

All members of **software development teams** should receive appropriate training to stay informed about **security and privacy basics** and recent **trends**

Phase One: Requirements



Opportunity to consider security at the outset of a project

- Development team **identifies security and privacy requirements**
- Development team identifies lead security and privacy contacts
- Security Advisor assigned
- Security Advisor reviews product plan, makes recommendations, may set additional requirements
- Mandate the use of a bug tracking/job assignment system
- Define and document security and privacy **bug bars**

Bug Bars and Quality Gates



- Bug Bars (and Quality Gates) are used to **establish minimum acceptable levels** of security and privacy quality
- **A Bug Bar is a Quality Gate** which is used to define the severity thresholds of security vulnerabilities
 - Example Bug Bar: no known vulnerabilities in the application with a “critical” or “important” rating at time of release.
 - Example Quality Gate: all compiler warnings must be triaged and fixed prior to code check-in
- The Bug Bar, **once set**, should **never be relaxed!**

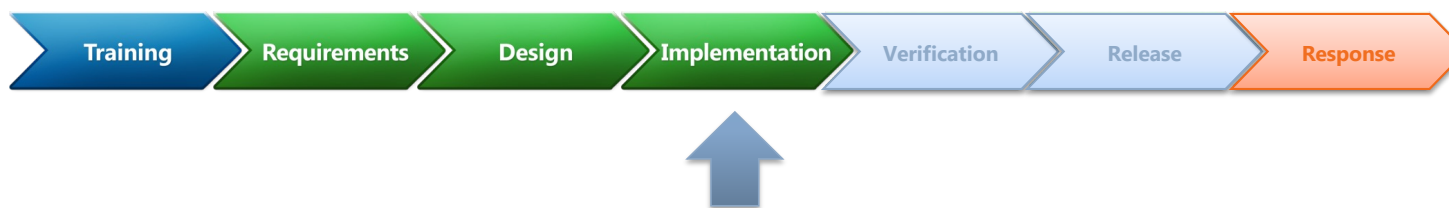
Phase Two: Design



Define and document security architecture, identify security critical components

- Identify **design techniques** (layering, managed code, least privilege, attack surface minimization)
- Document **attack surface** and limit through default settings
- Define supplemental security ship criteria due to **unique product issues**
 - Cross-site scripting tests
 - Deprecation of weak crypto
- **Threat Modeling**
 - Systematic **review of features** and product **architecture** from a security point of view
 - **Identify threats and mitigations**
- **Online services specific requirements**

Phase Three: Implementation



Full spectrum review – used to determine processes, documentation and tools necessary to ensure secure deployment and operation

- Specification of approved build tools and options
- Static analysis (Tools: PREFix, /analyze (PREfast), FXCop)
- Banned APIs
- Use of operating system “defense in depth” protections (NX, ASLR and HeapTermination)
- Online services specific requirements (e.g., Cross-site scripting , SQL Injection etc)
- Consider other recommendations (e.g., Standard Annotation Language (SAL))

Phase Four: Verification



Started as early as possible – conducted after “code complete” stage

- Start **security response planning** – including response plans for vulnerability reports
- Re-evaluate **attack surface**
- **Fuzz testing** – files, installable controls and network facing code
- Conduct “security push” (as necessary, increasingly rare)
 - Not a substitute for security work done during development
 - **Code review**
 - **Penetration testing** and other security testing
 - Review design and architecture in light of **new threats**
- **Online services** specific requirements



Phase Five: Release – Response Plan



Creation of a clearly defined support policy

- Provide Software Security Incident Response Plan (SSIRP)
 - **Identify contacts for Incident Response Team** and resources to respond to events
 - 24x7x365 contact information for 3-5 engineering, 3-5 marketing, and 1-2 management (PUM and higher) individuals
- Ensure ability to service all code including “out of band” releases and all licensed 3rd party code.



Phase Five: Release – Final Security Review (FSR)



Verify SDL requirements are met and there are no known security vulnerabilities

- Provides an independent view into “security ship readiness”
- The FSR is NOT:
 - A penetration test – no “penetrate and patch” allowed
 - The first-time security is reviewed
 - A signoff process
 - Key Concept: The tasks for this phase are used as a **determining factor on whether or not to ship** – not used as a “catchall” phase for missed work in earlier phases

Phase Five: Release - Archive



Security response plan complete

- Customer documentation up-to-date
- Archive RTM source code, symbols, threat models to a central location
- Complete final signoffs on Checkpoint Express - validating security, privacy and corporate compliance policies

Post-SDL Requirement: Response



“Plan the work, work the plan...”

- Execution on response tasks outlined during **Security Response Planning** and Release Phases



SDL Process Guidance for LOB Apps

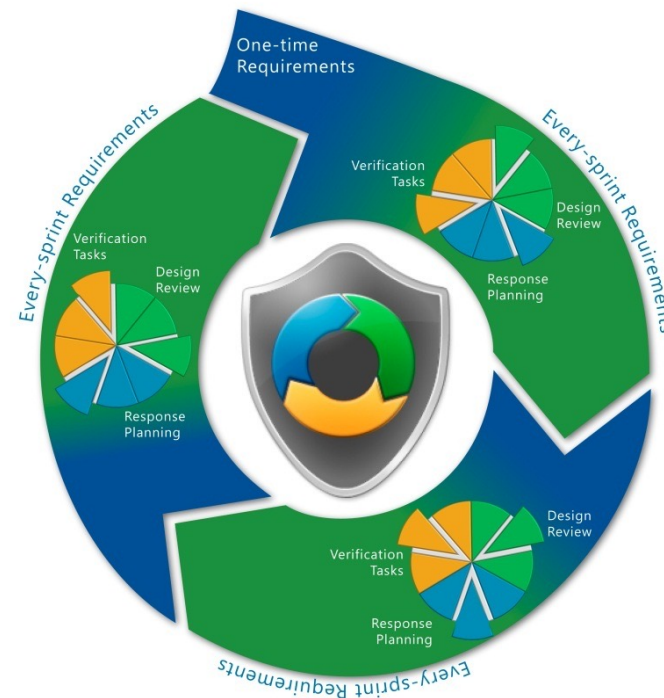
The Microsoft SDL includes online services and Line-of-Business application development guidance.

- Line-of-Business applications are a set of critical computer applications that are vital to running an enterprise, such as accounting, human resources (HR), payroll, supply chain management, and resource planning applications.
- Many of the requirements and recommendations in the SDL for online services are closely related to what is required for Line-of-Business applications.
- Line-of-Business SDL process guidance allows you to tailor a process specific to your LOB application development while meeting SDL requirements.

Training	Requirements	Design	Implementation	Verification	Release
LOB-specific training	Risk assessment • Application portfolio • Application Risk assessment • Determine service level	Asset-centric threat modeling • Threat model • Design review	Internal review • Incorporate security checklists and standards • Conduct self code review • Security Code analysis	Pre-production assessment • Comprehensive security assessment • Bug remediation	Post-production assessment • Host level scan

AGILE: SDL Guidance for Agile Methodologies

- Requirements defined by frequency, not phase
 - Every-Sprint (most critical)
 - One-Time (non-repeating)
 - Bucket (all others)
- Great for projects without end dates, like cloud services



SDLC - Requires Process Improvement



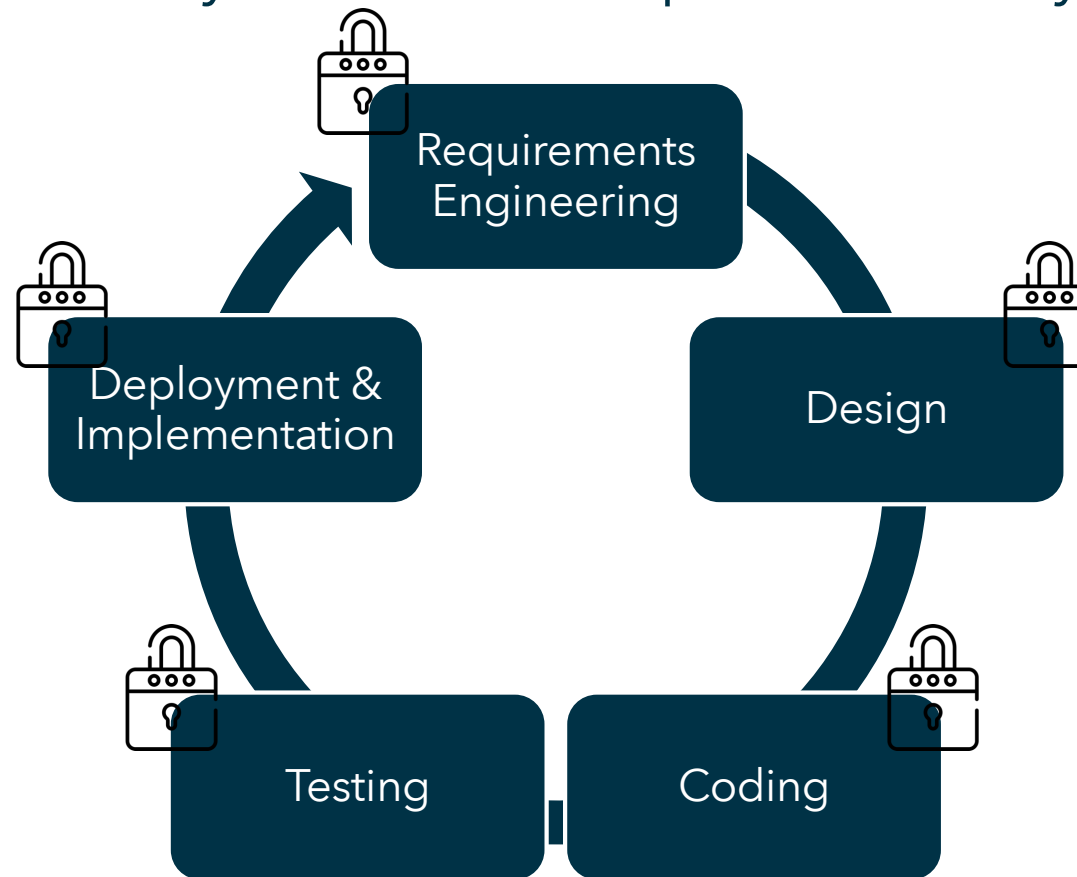
- Secure software development requires continuous process improvement
- Key Concepts
 - Simply “looking for bugs” doesn’t make software secure
 - Must reduce the chance vulnerabilities enter into design and code
 - Requires executive commitment
 - Requires ongoing process improvement
 - Requires education & training
 - Requires tools and automation
 - Requires incentives and consequences

Many Ways lead to Rome..



- Different concepts but at the end the same idea
- Most seen:
 - NIST
 - MSSDLC
 - OWASP SAMM

Integrating Security in the Development Life Cycle





Threat Modeling

Threat Analysis

Threat Modeling



- Threat modeling is a procedure for optimizing network security by identifying objectives and vulnerabilities, and then defining countermeasures to prevent, or mitigate the effects of, threats to the system.

<https://searchsecurity.techtarget.com/definition/threat-modeling>

- A structured method to analyze **assets** worth of protection, to analyze possible **attacks** on this assets, and to evaluate the **risk**. Based on this analysis **countermeasures (mitigations)** are defined and specified as requirements.

Shostack, Adam: "Threat Modeling: Designing for Security". Wiley, 2014



What's important

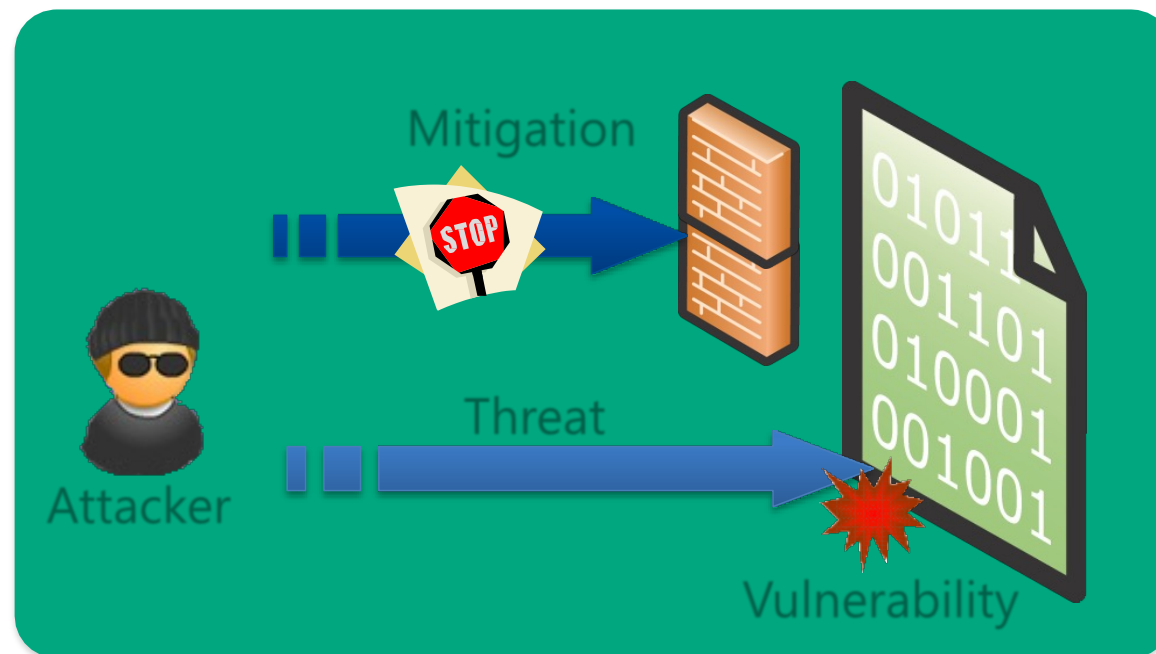
- To be secure (design, develop, implement)
 - Build an understanding of the threats
 - Stay up2date
 - Knowledge of common attack vectors / scenarios
 - Differ between threats and vulnerabilities
 - Threats won't fade away
 - Threats exist forever and are therefore the attackers goal

THREAT VERSUS VULNERABILITY

Threat is a person or thing likely to cause damage or danger	Vulnerability refers to being open to attack or damage
Danger posed by someone else	Flaw or weakness in us
Can be identified, but cannot be controlled	Can be identified and corrected
	Pediaa.com

<http://pediaa.com/difference-between-threat-and-vulnerability/>

Threats are not vulnerabilities



Vulnerability - Threat - Threat - Action - Risk



Vulnerability	Threat	Threat Action	Risk
A weakness in a security program that can be exploited BY a threat to gain unauthorized/unc ontrolled access to an asset	Anything that can exploit a vulnerability (intentionally or accidentally) and damage, obtain or even destroy an asset	The final assault on system security that the threat cause (intentional or accidental)	The potential (%) for damage , loss or destruction of an asset as a result of a threat exploiting a vulnerability



Ex: Vulnerability – Threat – Action

Vulnerability	Threat	Mitigation (Action)
Firewall not up2date	Unauthorized Connections	Update, hardening FW
User won't be authenticated correctly	?	?

Threat Modeling



- Threat modeling is an important part of any **system life-cycle** and especially the **design part**.

It will allow the

- designers
- Security experts
- team
- management

to **determine** what could cause a possible threat to the system.

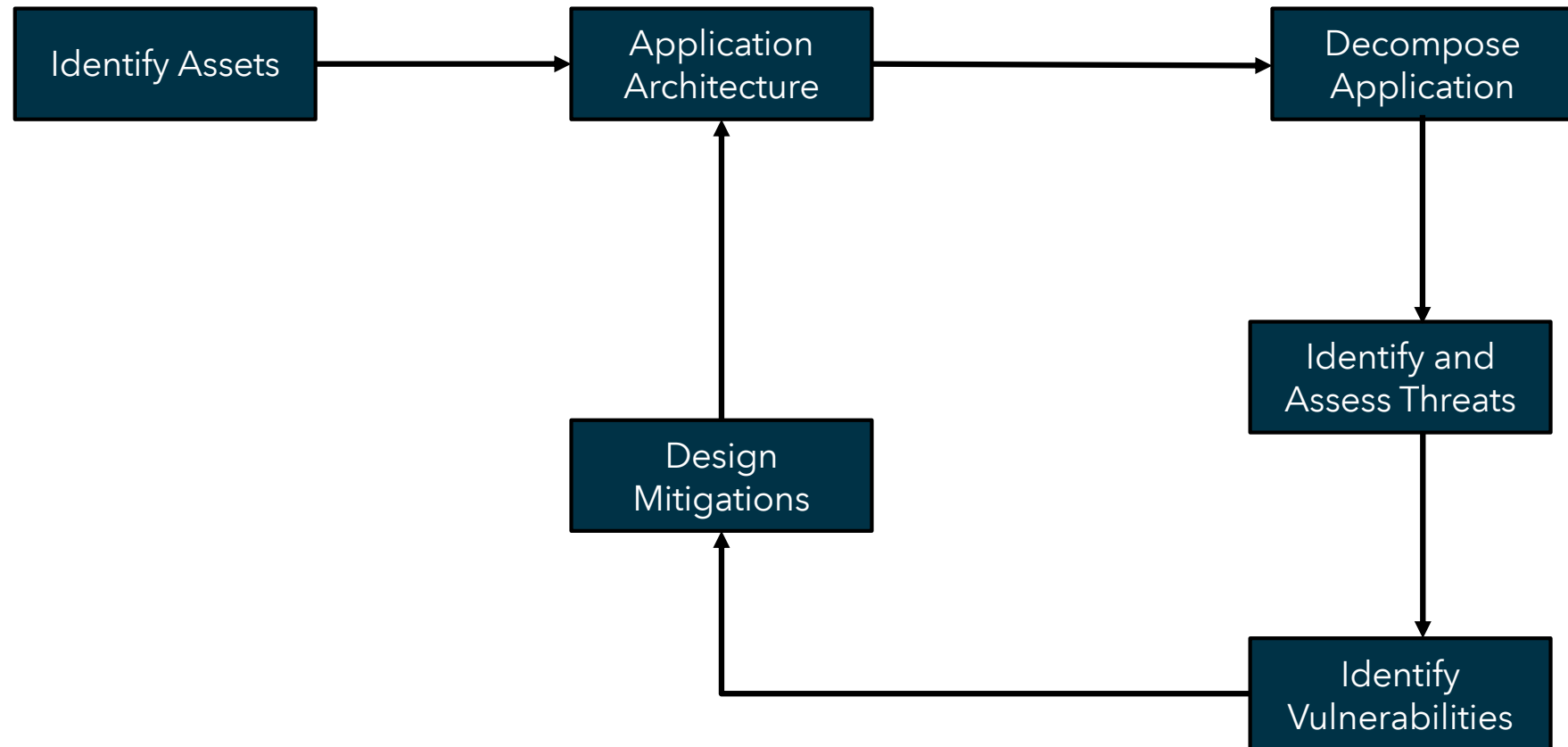
- This is done by taking all possibilities even the small ones, analyze and prioritize them and come up with a **mitigation plan** for each.

Why Threat Modeling?



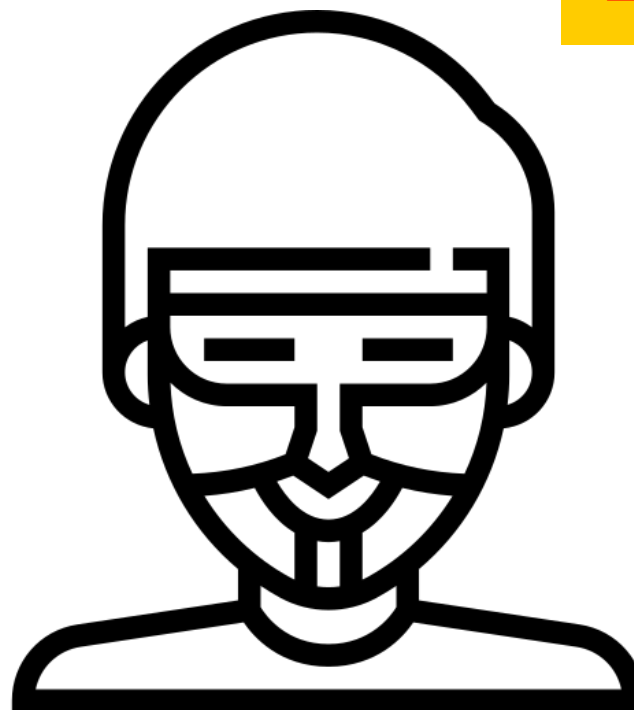
- It is better to find security flaws when there is time to fix them.
- It can save time, revenue and the reputation of your company.
- To build a secure application.
- To bridge the gap between developers and security.
- It provides a document of all the identified threats and rated threats.
- It offers knowledge and awareness of the latest risks and vulnerabilities.

Threat Modeling → Iterations & Feedback to Arch.



Change your view

- Where would you attack to harm the assets of any of the involved parties?
- Brainstorming!
- Take the attacker's view!



Threat Modeling



Asset

- the object we need to protect
- Value may vary from context to context
- Can be tangible or intangible

Vulnerability

- Weaknesses in the system
- a lack of or a wrong countermeasure
- Can be everywhere

Threat

- any potential danger associated with a vulnerability
- someone, or something, will attack the system

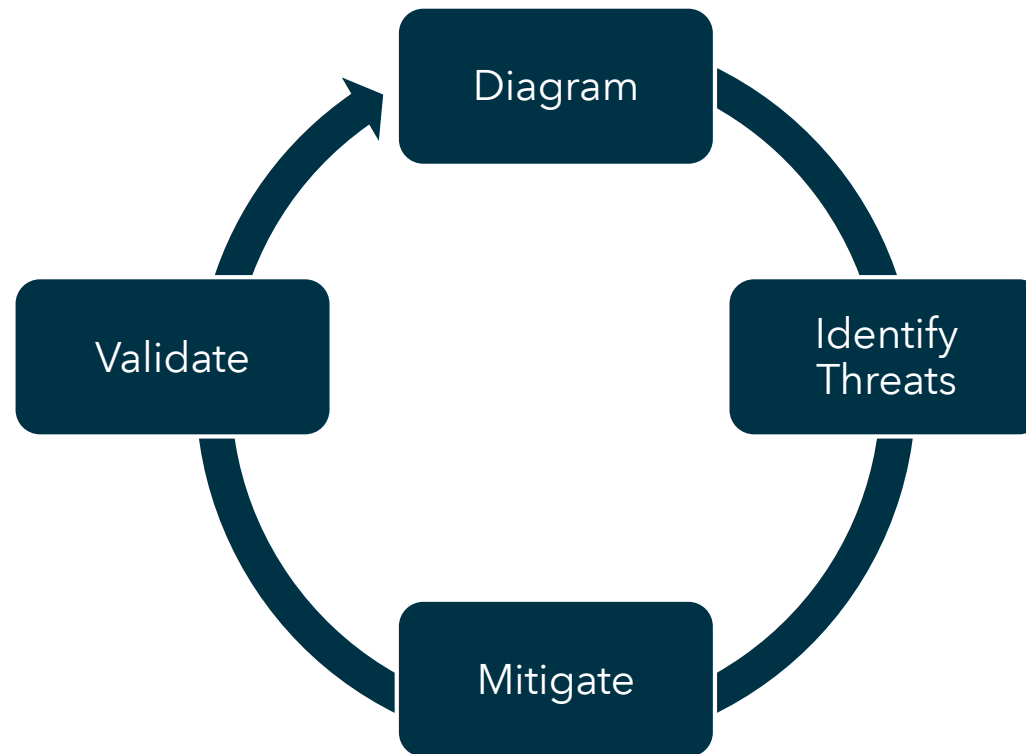
Exploit

- When a threat becomes reality → Attack

Mitigate

- Limiting the impact or likelihood of an attack

The Process of Threat Modeling

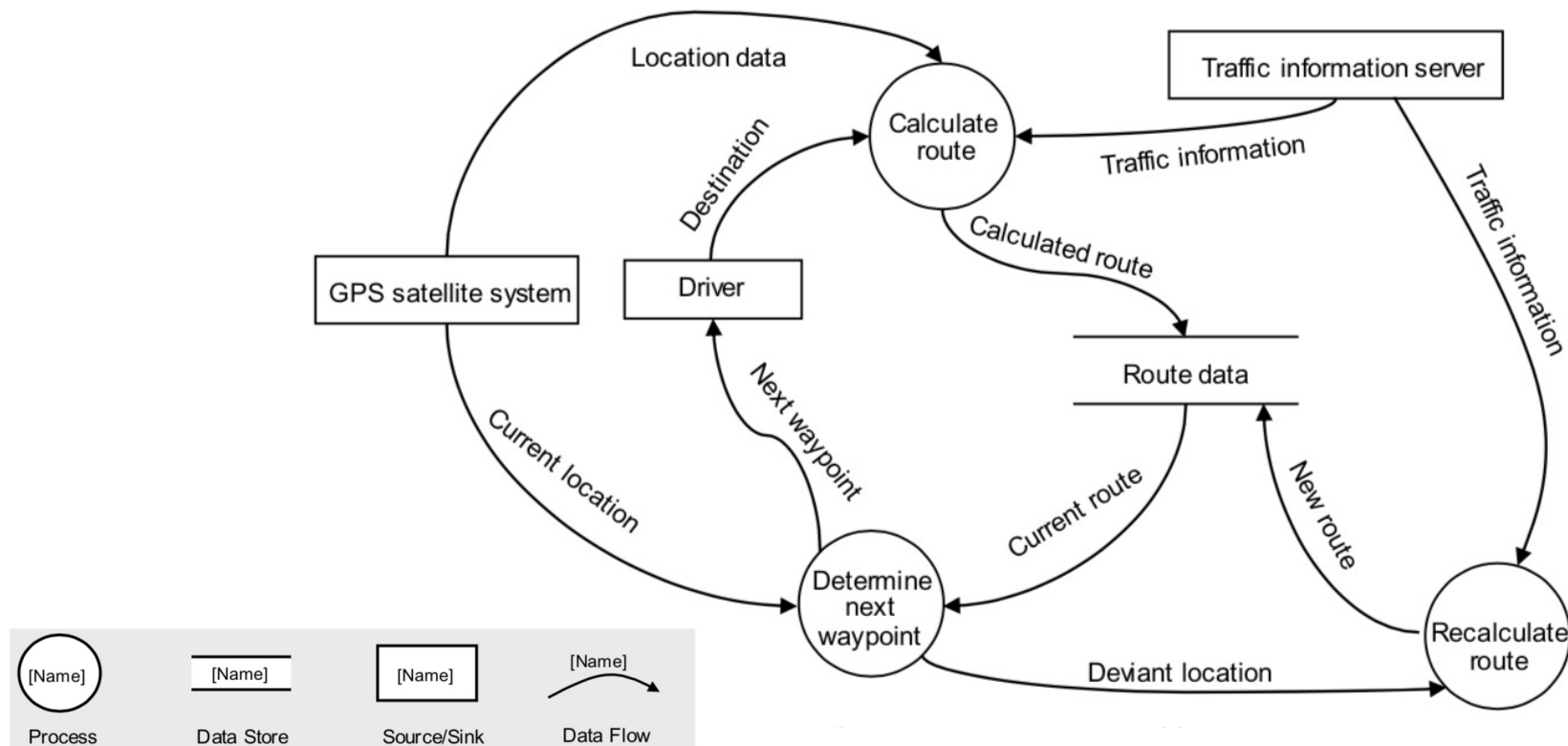


Diagramming

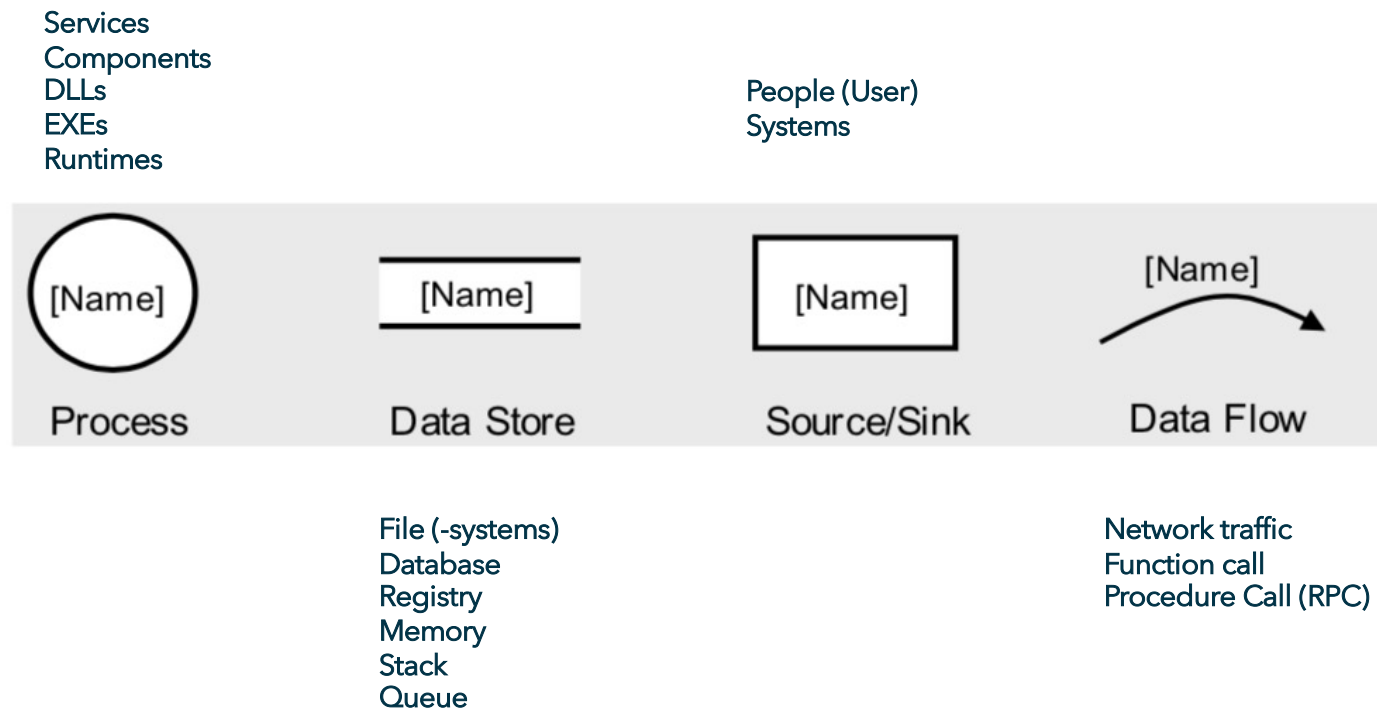


- Use DFDs (Data Flow Diagrams)
 - Include processes, data stores, data flows
 - Include trust boundaries
 - Diagrams per scenario may be helpful
- Update diagrams if the product changes
- Enumerate everything (if manual)

Remember: Dataflow diagram (DFD)



Notation of DFD



Create Diagrams



- Start with an overview which has:
 - A few external interactors
 - One or two processes
 - One or two data stores
 - Data flows to connect them
- Check your work
 - Can you tell a story without edits?
 - Does it match reality?

Diagrams: Trust Boundaries



- Add trust boundaries that intersect data flows
- Point/surface where an attacker can interject
 - Machine boundaries, privilege boundaries, integrity boundaries are examples of trust boundaries
 - Threads in a native process are often inside a trust boundary, because they share the same
 - privileges, rights, identifiers and access
- Processes talking across a network always have a trust boundary
 - They may create a secure channel, but they're still distinct entities
 - Encrypting network traffic is an 'instinctive' mitigation, but doesn't address tampering or spoofing

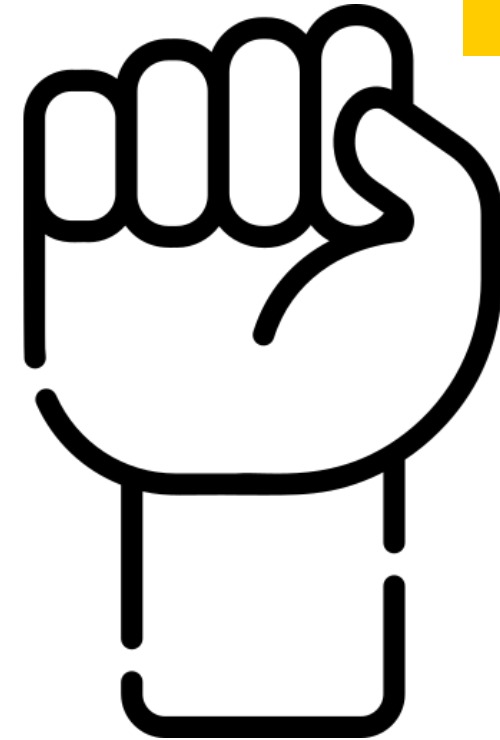
Diagram Iteration



- Iterate over processes, data stores, and see where they need to be broken down
- How to know it “needs to be broken down”
 - More detail is needed to **explain security impact** of the design
 - Object crosses a **trust boundary**
 - Words like “sometimes” and “also” indicate you have a combination of things that can be broken out
 - “Sometimes this data store is used for X”... probably add a second data store to the diagram

Rules of thumb - Cheat Sheet

- Does data magically appear?
- Are there data sinks?
- Check: Data doesn't flow magically
- Check: "It" goes through a process



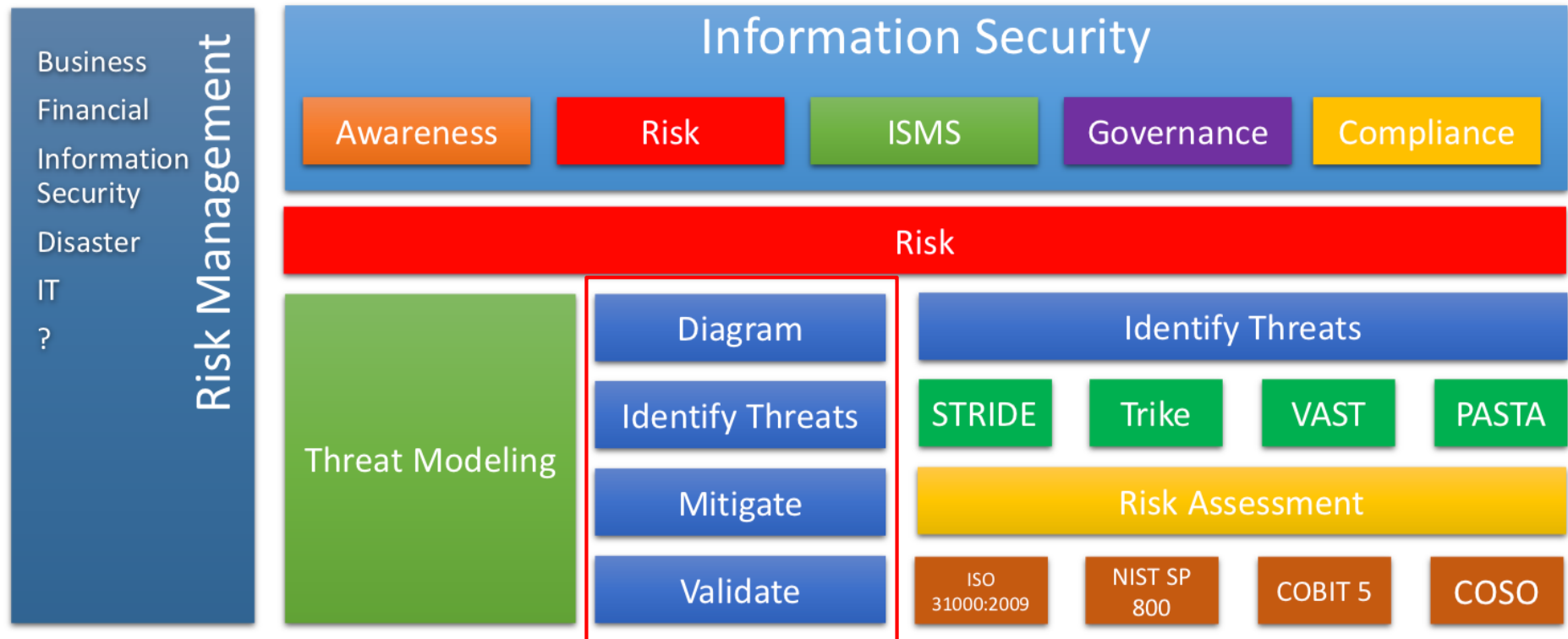
Threat Modeling



Brainstorming support. How you get started...

- STRIDE attack classification
- OWASP Threat List
- Attack trees

The Big Picture



STRIDE and DREAD



- When modeling the system threats, we need to **classify the possible threats** that we found in order to better **understand what attackers can do**.
- We need to determine what **level of risks their threats** can pose by applying a certain measurement mechanism, **quantifying the risks** which will help us in the **prioritization** process.



What can go wrong in this system we're working on?

Security Testing Principles



- **Confidentiality:** This is equivalent to privacy, and it has a set of rules which limits access to information. It protects against disclosure of information to unintended recipients, and is designed to prevent sensitive information from reaching the wrong people. It ensures that only the designated person gets the information and access will be restricted to those authorized to view the data in question.
- **Integrity:** It involves maintaining the consistency, accuracy, and trustworthiness of data over its entire life cycle, and allows transferring accurate and desired information from senders to intended receivers. It ensures that data cannot be altered by unauthorized people (for example, in a breach of confidentiality).
- **Authentication:** This confirms the identity of a user and allows a user to have confidence that the information he receives originated from specific known sources.
- **Authorization:** It specifies access rights to the users, based on the user role.
- **Availability:** Ensures the readiness of the information on requirement. To simplify, information must be available to authorized person(s) when they require it. Availability is best ensured by rigorously maintaining all hardware, performing hardware repairs immediately when needed and maintaining a correctly functioning operating system environment that is free of software conflicts. [ref: <http://whatis.techtarget.com/definition/Confidentiality-integrity-and-availability-CIA>]
- **Non-repudiation:** This ensures there is no denial from the sender or the receiver for sent /received messages. It exchanges authentication information with provable time stamp, for example, 'session id' and so forth.



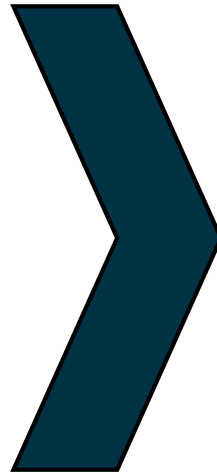
Each threat is a violation of a desirable property for a system!

Identify/Enumerate Threats



Threat

- Spoofing
- Tampering
- Repudiation
- Information Disclosure
- Denial of Service
- Elevation of Privilege



Property we want

- Authentication
- Integrity
- Nonrepudiation
- Confidentiality
- Availability
- Authorization

Use STRIDE to step through the diagram elements

THREAT: Spoofing



Threat	Spoofing
Property	Authentication
Definition	Impersonating something or someone else
Example	Pretending to be any message of CEO, Customer or Service etc.

Spoofing



- Phone Authentication
 - SMS or calls (Your bank calls you)
 - Scamicry
 - The caller demands authentication from callee
 - Phone porting attacks
- Facial Recognition
 - Selfies are sold on the darkweb (fullz)

Example Spoofing

- Facial Recognition
- Deepfake
- UK-Krieg
- <https://www.zdf.de/nachrichten/politik/selenskyj-deepfake-video-ukraine-krieg-russland-100.html>



<https://www.youtube.com/watch?v=bdLQ-NDy5WA>
www.security.ch

THREAT: Tampering



Threat	Tampering
Property	Integrity
Definition	Modify data or code
Example	Modifying a DLL on disk or a data-packet

Update: Tampering



- Changing Libraries
 - Statscounter and /account/withdraw/btc
 - <https://www.zdnet.com/article/hackers-breach-statcounter-to-hijack-bitcoin-transactions-on-gate-io-exchange/>
- "AirBNB Attacker" tamper a smart lock
 - <https://youtu.be/RxM55DNS9CE>
 - https://www.theregister.co.uk/2018/06/15/taplock_broken_screwdriver/

Video – AirBnB Hack



THREAT: Repudiation



Threat	Repudiation
Property	Non-Repudiation
Definition	Claiming to have not performed an action
Example	"I didn't send that email," "I didn't modify that file," "I certainly didn't visit that Web site!"

Repudiation

- Migrating Storks racks up to 2700\$ phone bill
 - <https://www.iflscience.com/plants-and-animals/migrating-stork-racks-up-2700-on-researchers-cell-phone-bill/>



THREAT: Information Disclosure



Threat	Information Disclosure
Property	Confidentiality
Definition	Exposing information to a unauthorized target
Example	Publishing confidential data (customer list) to a website

Information Disclosure

- Location based services
 - Exposed US military bases
 - →DOD Ban
- Lots of other sensors that collect information and may publish it to someone
- <https://www.wired.com/story/strava-heat-map-military-bases-fitness-trackers-privacy/>



<https://twitter.com/Nrg8000/status/957318498102865920>

THREAT: Denial of Service



Threat	DoS
Property	Availability
Definition	Deny or degrade (to a unusable level) service to users
Example	Sending lots of requests to a website

Denial of Service

- Not just take you out of service anymore
 - Sending lots of requests (even distributed - D-dos)
- Use your computer power for their needs (i.e. crypto mining)



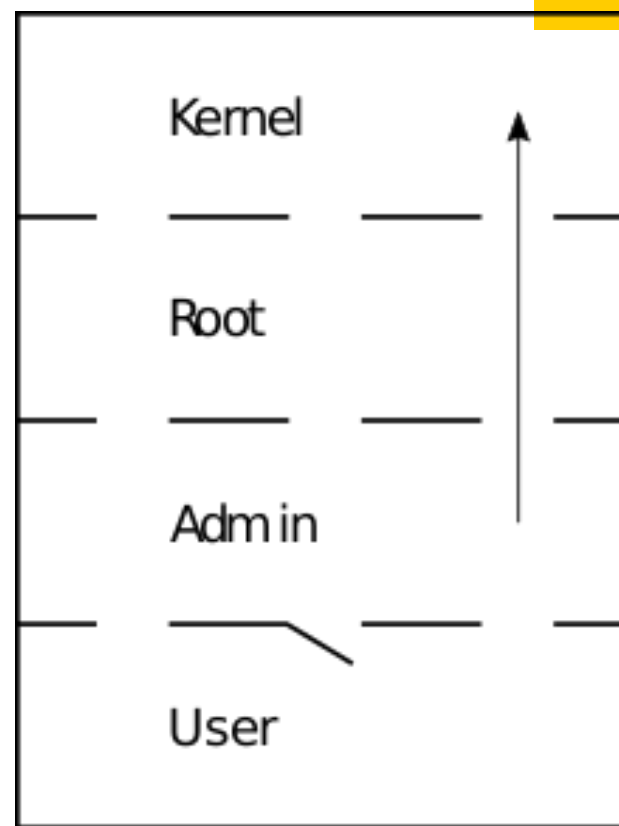
THREAT: Elevation of Privilege



Threat	Elevation of Privilege
Property	Authorization
Definition	Gain roles without a proper authorization
Example	Allowing user to run "sudo" commands (std User → root)

Elevation of Privileges

- Many isolation breaks
 - Lightning cables
 - Docker / Kubernetes escapes
 - Spectre / Meltdown on processors
 - <https://meltdownattack.com>
 - <https://arxiv.org/pdf/1902.05178.pdf>



STRIDE Example



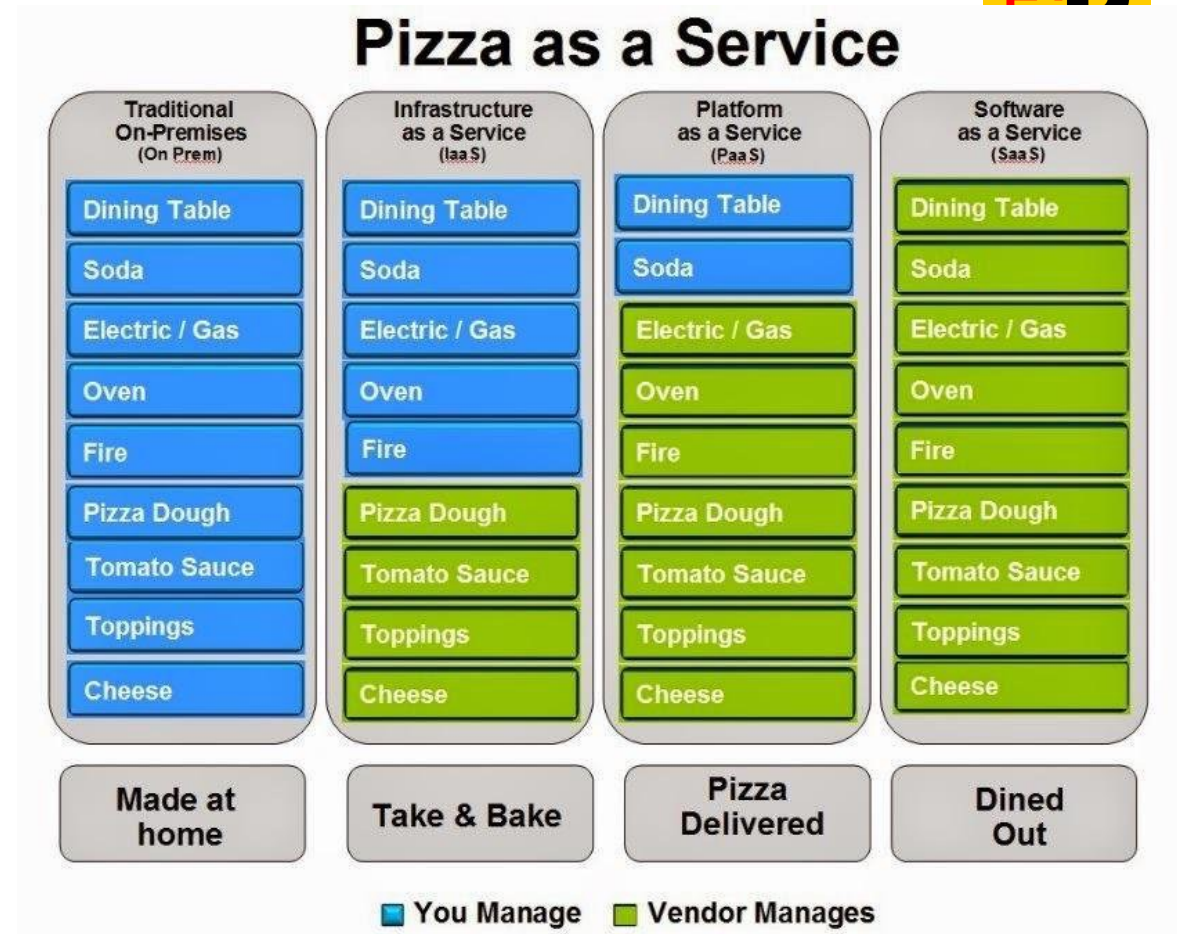
STRIDE Example

- Spoofing `I am Spartacus.`
- Tampering `Looks like Johnny got an A!`
- Repudiation `Didn't Johnny get a B?`
- Information disclosure `Johnny's SSN is...`
- Denial of Service `Please try again later.`
- Elevation of privilege `sudo rm -rf /home/johnny`

Secure Coding | 2017 | Richard Thron

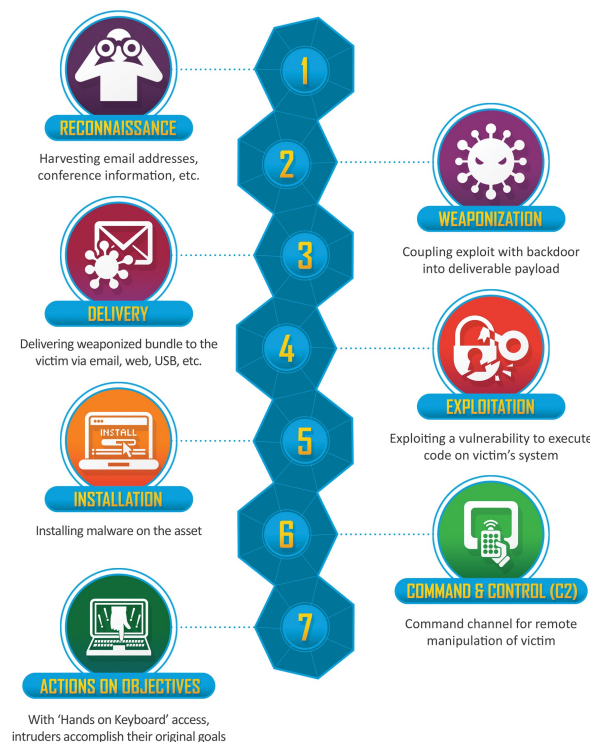
Challenges with Stride

- “old” - >20 years
- Still helpful mnemonic
- Moving to cloud
 - XY – aaS
 - Serverless



Kill Chain as alternative to Stride?

- Attack and CK is a very granular model of what attackers do after they break in.
- STRIDE is a general model of what attackers do to break software.
- In military parlance, a "kill chain" is a **phase-based model** to describe the stages of an attack, which also helps inform ways to prevent such attacks. These stages are referred to as:
 - Find, Fix, Track, Target, Engage, Assess
- The closer to the beginning of the kill chain an attack can be stopped, the better. The less information an attacker has, for instance, the less likely someone else can use that information to complete the attack later.



Mobile Payment Usecase



© GfK 2015

Threat Modeling Tools



- Microsoft Threat Modeling Tool 2016
 - <https://www.microsoft.com/en-us/download/details.aspx?id=49168>
- OWASP Threat Dragon
 - https://www.owasp.org/index.php/OWASP_Threat_Dragon